

Algorithms for the Third Cycle: Two-Pass Assembler and Absolute Loader

October 29, 2025

Algorithm 1: First Pass of the Two-Pass Assembler

Input: SIC source file

Output: intermediate file

```
1: read first input line
2: if OPCODE = START then
3:   STARTADDR  $\leftarrow$  OPERAND
4:   LOCCTR  $\leftarrow$  OPERAND
5:   write line to intermediate file
6:   read next line
7: else
8:   LOCCTR  $\leftarrow$  0
9: end if
10: while OPCODE  $\neq$  END do
11:   if LABEL exists then
12:     search SYMTAB for LABEL
13:     if found then
14:       set error flag                                 $\triangleright$  duplicate symbol
15:     else
16:       insert (LABEL, LOCCTR) into SYMTAB
17:     end if
18:   end if
19:   search OPTAB for OPCODE
20:   if found then
21:     LOCCTR  $\leftarrow$  LOCCTR + 3
22:   else if OPCODE = WORD then
23:     LOCCTR  $\leftarrow$  LOCCTR + 3
24:   else if OPCODE = RESW then
25:     LOCCTR  $\leftarrow$  LOCCTR + 3  $\times$  OPERAND
26:   else if OPCODE = RESB then
27:     LOCCTR  $\leftarrow$  LOCCTR + OPERAND
```

```

28:   else if OPCODE = BYTE then
29:       LOCCTR  $\leftarrow$  LOCCTR + StrLen(OPERAND)
30:   else
31:       set error flag                                 $\triangleright$  invalid operation code
32:   end if
33:   write line to intermediate file
34:   read next input line
35: end while
36: write last line to intermediate file
37: save LOCCTR – STARTADDR as program length

```

Algorithm 2: Second Pass of the Two-Pass Assembler

```

Input: intermediate file
Output: object program

```

```

1: read first input line
2: write Header record
3: initialize first Text record
4: while OPCODE  $\neq$  END do
5:     search OPTAB for OPCODE
6:     if found then
7:         if OPERAND exists then
8:             search SYMTAB for OPERAND
9:             if found then
10:                store symbol value as operand address
11:            else
12:                store 0 as operand address
13:                set error flag                                 $\triangleright$  undefined symbol
14:            end if
15:        else
16:            store 0 as operand address
17:        end if
18:        assemble the object code instruction
19:    else if OPCODE = BYTE  $\vee$  OPCODE = WORD then
20:        convert OPERAND to object code
21:        if object will not fit into the Text record then
22:            write Text record
23:            initialize new Text record
24:        end if
25:    end if
26:    add object code to Text record
27:    write listing line
28:    read next input line

```

29: **end while**
30: write last **Text** record
31: write **End** record
32: write last listing line

Algorithm 3: Absolute Loader

Input: object program

Output: program in memory

1: read **Header** record
2: verify program name and length
3: read first **Text** record
4: **while** not an **End** record **do**
5: convert object code into internal representation
6: move object code to specified location in memory
7: read next record
8: **end while**
9: jump to address specified in **End** record
